

Generative AI Lab

Programs with Line-by-Line Explanations

Labs 1 – 7 | Labs 9 – 10

(Lab 8 excluded as per requirement)

Lab 1: Word Embeddings – Analogy & Similarity (GloVe)

Loads a pre-trained 50-dim GloVe model and performs word vector arithmetic analogies.

```
1■ from gensim.downloader import load # Import gensim downloader to fetch pre-trained embedding models
2■
3■
4■ print("Loading pre-trained GloVe model (50 dimensions)...") # Print a loading message – model download
can take a while
5■ model = load("glove-wiki-gigaword-50") # Load the 50-dim GloVe model trained on Wikipedia + Gigaword
corpus
6■
7■
8■ def ewr(): # Define function ewr() that runs all embedding analogy experiments
9■     result = model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1) # Vector math:
king - man + woman → nearest word should be 'queen'
10■     print("\nking - man + woman = ?", result[0][0]) # Print the top-1 analogy result word
11■     print("similarity:", result[0][1]) # Print the cosine similarity score (0-1) of the result
12■
13■     result = model.most_similar(positive=['paris', 'italy'], negative=['france'], topn=1) # Geography
analogy: paris - france + italy → expected answer 'rome'
14■     print("\nparis - france + italy = ?", result[0][0]) # Print the capital-of-Italy analogy result
15■     print("similarity:", result[0][1]) # Print its similarity score
16■
17■     result = model.most_similar(positive=['programming'], topn=5) # Find the 5 words closest to
'programming' in the embedding space
18■     print("\nTop 5 words similar to 'programming':") # Print header for the nearest-neighbour results
19■     for word, similarity in result: # Iterate over each (word, score) pair in the result
20■         print(word, similarity) # Print each similar word and its cosine similarity value
21■
22■
23■ ewr() # Call ewr() to execute all the analogy and similarity experiments
```

Lab 2: Word Embedding Visualisation with PCA

Fetches GloVe vectors for 10 tech words, reduces them to 2-D with PCA, and plots them.

```
1■ import matplotlib.pyplot as plt # Import matplotlib for scatter-plot visualisation
2■ from sklearn.decomposition import PCA # Import PCA to reduce 50-D vectors to 2-D
3■ from gensim.downloader import load # Import gensim downloader
4■
5■
6■
7■ def rd(ems): # Function rd(): fits PCA on embeddings and returns 2-D coordinates
8■     pca = PCA(n_components=2) # Create PCA object targeting 2 principal components
9■     r = pca.fit_transform(ems) # Fit PCA on the embedding matrix and project to 2-D
10■     return r # Return the 2-D reduced array
11■
12■
13■
14■ def visualize(words, ems): # Function visualize(): scatter-plots each word at its 2-D position
15■     plt.figure(figsize=(10, 6)) # Create a 10x6 inch figure canvas
16■     for i, word in enumerate(words): # Iterate over words with their index for annotation
17■         x, y = ems[i] # Unpack the 2-D (x, y) coordinate for the current word
18■         plt.scatter(x, y, marker='o', color='blue') # Draw a blue dot at the word's position in 2-D
space
19■         plt.text(x + 0.02, y + 0.02, word, fontsize=12) # Label the dot with the word name, offset
slightly so it doesn't overlap
20■     plt.show() # Render and display the completed scatter plot
21■
22■
23■ def gsm(word): # Function gsm(): prints top-5 nearest neighbours of a given word
24■     sw = model.most_similar(word, topn=5) # Retrieve 5 closest words in embedding space
25■     for word, s in sw: # Iterate over (word, score) results
26■         print(word, s) # Print each neighbour and its similarity
27■
28■
29■ print("Loading pre-trained GloVe model (50 dimensions)...") # Load the 50-dim GloVe model
30■ model = load("glove-wiki-gigaword-50")
31■ tech_words = ["computer", "algorithm", "software", "hardware", "code", # Define list of 10
technology-domain words to analyse
32■ "cloud", "database", "network", "cybersecurity", "encryption"]
33■
34■ ems = [model[word] for word in tech_words] # Fetch the 50-D embedding vector for each word in the
list
35■ e = rd(ems) # Reduce all embeddings from 50-D to 2-D using PCA
36■ visualize(tech_words, e) # Plot the 2-D word positions
37■ gsm("hardware") # Print 5 nearest neighbours of 'hardware'
```

Lab 3: Custom Word2Vec on Medical Corpus

Trains a Word2Vec model from scratch on 10 medical sentences, then visualises embeddings with PCA.

```
1 from gensim.models import Word2Vec # Import Word2Vec model class from gensim
2 from sklearn.decomposition import PCA # Import PCA for 2-D projection
3 import matplotlib.pyplot as plt # Import matplotlib for plotting
4
5
6 corpus = [ # Define 10 medical sentences as the raw training corpus
7 "The patient was diagnosed with diabetes and hypertension.",
8 "MRI scans reveal abnormalities in the brain tissue.",
9 "The treatment involves antibiotics and regular monitoring.",
10 "Symptoms include fever, fatigue, and muscle pain.",
11 "The vaccine is effective against several viral infections.",
12 "Doctors recommend physical therapy for recovery.",
13 "The clinical trial results were published in the journal.",
14 "The surgeon performed a minimally invasive procedure.",
15 "The prescription includes pain relievers and anti-inflammatory drugs.",
16 "The diagnosis confirmed a rare genetic disorder."
17 ]
18
19
20 token_corp=[sentence.lower().split() for sentence in corpus] # Tokenise corpus: lowercase each
sentence and split into word list
21 model=Word2Vec(sentences=token_corp,vector_size=5>window=2, min_count=1,epochs=1000) # Train
Word2Vec: 5-D vectors, window=2, min_count=1, 1000 epochs
22 w=input("enter a word:").lower() # Ask user for a query word and convert to lowercase
23
24
25 if w in model.wv: # Check if the word exists in the trained vocabulary
26     similar=model.wv.most_similar(w,topn=5) # Find the 5 most similar words in the trained embedding
space
27     print(f" word similar to {w}")
28     for i ,(wo,score) in enumerate(similar,1): # Iterate with 1-based rank index
29         print(f"{i}.{wo} similarity:{score} ") # Print rank, similar word, and similarity score
30 else:
31     print(" word not found in the vocabulary") # Inform user if queried word is not in the vocabulary
32
33
34 words=list(model.wv.index_to_key) # Get all vocabulary words as an ordered list
35 word_vectors=model.wv[words] # Retrieve the full embedding matrix for all vocabulary words
36 pca=PCA(n_components=2) # Create PCA object for 2 output dimensions
37 result=pca.fit_transform(word_vectors) # Project all word vectors to 2-D
38 plt.figure(figsize=(20,8)) # Create a wide 20x8 inch figure
39 plt.scatter(result[:,0],result[:,1]) # Scatter-plot every word at its 2-D PCA coordinates
40
41 for i,word in enumerate(words): # Iterate over words for text annotation
42     plt.annotate(word,xy=(result[i,0],result[i,1])) # Label each scatter point with the corresponding
word
43
44
45 plt.title("word embeddings visualization") # Set chart title
46 plt.xlabel("PCA 1") # Label x-axis as first principal component
47 plt.ylabel("PCA 2") # Label y-axis as second principal component
48 plt.grid(True) # Show grid lines for easier reading
49 plt.show() # Render and display the plot
```

Lab 4: Prompt Enrichment with GloVe + GPT-2 Text Generation

Replaces a target keyword in a prompt with its closest GloVe neighbour, then compares GPT-2 outputs.

```
1 from gensim.downloader import load # Import gensim downloader for pre-trained word vectors
2 from transformers import pipeline # Import HuggingFace pipeline helper for text generation
3 import nltk # Import nltk for tokenisation and string for punctuation constants
4 import string
5 from nltk.tokenize import word_tokenize # Import NLTK's word-level tokeniser
6 nltk.download('punkt_tab') # Download Punkt tokeniser data if not already installed
7 print("loading pre trained word vectors") # Print loading notice for the GloVe model
8 word_vectors=load("glove-wiki-gigaword-100") # Load 100-dim GloVe embeddings (richer than 50-dim)
9
10 def replace_keyword_in_prompt(prompt,keyword,word_vectors,topn=1): # Define function to replace a
keyword with its nearest GloVe word
11     words=word_tokenize(prompt) # Tokenise the prompt string into individual word tokens
12     enriched_words=[] # Initialise list to hold the modified token sequence
13     for word in words: # Loop over each token in the prompt
14         cleaned_word=word.lower().strip(string.punctuation) # Lowercase and strip punctuation for clean
keyword matching
15         if cleaned_word==keyword.lower(): # Check if this token matches the target keyword
16             try:
17                 similar_words=word_vectors.most_similar(cleaned_word,topn=topn) # Fetch the single closest
GloVe word for the keyword
18                 if similar_words: # Check a similar word was actually found
19                     replacement_word=similar_words[0][0] # Take the top-ranked similar word as the replacement
20                     print(f"Replacing {word}-> {replacement_word}") # Log the substitution being made
21                     enriched_words.append(replacement_word) # Add the replacement word to the output token list
22                     continue # Skip appending the original keyword word
23             except KeyError: # Handle case where keyword is not in the GloVe vocabulary
24                 print(f"{keyword} not found in vocabulary using original word")
25                 enriched_words.append(word) # Fall back to keeping the original word
26
27     enriched_prompt=" ".join(enriched_words) # Re-join the modified tokens back into a prompt string
28     print(f"\n Enriched Prompt:{enriched_prompt}") # Print the enriched prompt for inspection
29     return enriched_prompt # Return the enriched prompt string
30
31
32 print("\n Loading GPT-4 model") # Print a notice before loading GPT-2
33 generator=pipeline("text-generation",model="gpt2") # Load GPT-2 text-generation pipeline from
HuggingFace
34
35 def generate_response(prompt,max_length=100): # Define function to generate text from any prompt
using GPT-2
36     try:
37         response=generator(prompt,max_length=max_length,num_return_sequences=1) # Call GPT-2 with max 100
tokens and request 1 output sequence
38         return response[0]['generated_text'] # Return the generated text string
39     except Exception as e: # Catch any inference errors gracefully
40         print(f"error generating response {e}") # Print the error message and return None
41         return None
42
43 original_prompt="write an essay on natural disaster" # Define the base prompt to test
44 print(f"Original prompt: {original_prompt}") # Print the original prompt
45 k_term="disaster" # Set 'disaster' as the keyword to replace
46 enriched_prompt=replace_keyword_in_prompt(original_prompt,k_term,word_vectors) # Create the enriched
prompt by substituting the keyword
47 print("\n generating response for original prompt") # Print label before generating original response
48 original_response=generate_response(original_prompt) # Generate GPT-2 text for the original prompt
49 print(original_response) # Print the original generated text
50
51 print("\n generating response for enriched prompt") # Print label before generating enriched response
```

```
52■ enriched_response= generate_response(enriched_prompt) # Generate GPT-2 text for the enriched prompt
53■ print(enriched_response) # Print the enriched generated text
54■
55■ print("\n comparison of responses") # Print section header for comparison metrics
56■ print("original prompt response length",len(original_response)) # Compare the character length of
both responses
57■ print("enriched prompt response length",len(enriched_response)) # Compare length of enriched response
58■ print("original prompt response detail",original_response.count(".")) # Count sentence-ending periods
as a detail-richness proxy
59■ print("enriched prompt response detail",enriched_response.count(".")) # Count periods in enriched
response
```

Lab 5: Story Generation using GloVe Word Similarities

Uses a seed word's 50 nearest GloVe neighbours to fill slots in a short story template.

```
1■ import random # Import random module for selecting words randomly from a list
2■ from gensim.downloader import load # Import gensim downloader
3■
4■
5■ print("loading the glove model") # Print loading message
6■ model = load("glove-wiki-gigaword-200") # Load the 200-dim GloVe model - larger vocab and richer
semantics
7■
8■ def get_similar_words(seed_word, top_n=50): # Define function to retrieve the top-N similar words for
a seed
9■     """Retrieve similar words for the given seed word."""
10■     try:
11■         similar_words = [word for word, _ in model.most_similar(seed_word, # Get top-N nearest
neighbours, discarding similarity scores
12■ topn=top_n)] # Continuation of the most_similar call (line wrapped)
13■         return similar_words # Return the list of similar words
14■     except KeyError: # Handle case where seed word is absent from the vocabulary
15■         return [] # Return empty list so caller can handle gracefully
16■
17■ def create_paragraph(seed_word): # Define function to build a paragraph-length story
18■     """Generate a short paragraph using the seed word and its similar words.""" # Docstring
describing the function's purpose
19■     similar_words = get_similar_words(seed_word) # Fetch similar words to use as story vocabulary
20■
21■     if not similar_words: # Return a friendly error message if no similar words found
22■         return f"Could not find similar words for '{seed_word}'. Try another word!" # End of fallback
message string
23■
24■     paragraph = f"""Once upon a time, a {seed_word} embarked on a journey. # Begin constructing the
story paragraph using an f-string template
25■ Along the way, it encountered a {random.choice(similar_words)}, which led it to a hidden
{random.choice(similar_words)}. # Fill first story slot with a random similar word
26■ Despite the challenges, it found {random.choice(similar_words)} and embraced the adventure with
{random.choice(similar_words)}. # Fill second and third story slots with random similar words
27■ In the end, the journey was a tale of {random.choice(similar_words)} and discovery.""" # Fill fourth
story slot with a random similar word
28■
29■     return paragraph # Return the completed story paragraph
30■
31■
32■ seed_word = input("Enter a seed word: ").strip().lower() # Prompt user to enter a seed word and
normalise to lowercase
33■ print("\nGenerated Story:\n") # Print a header before the generated story
34■ print(create_paragraph(seed_word)) # Generate and print the story paragraph
```

Lab 6: Sentiment Analysis with HuggingFace Transformers

Interactive loop that classifies the sentiment of user-entered sentences using a pre-trained transformer.

```
1■ from transformers import pipeline # Import HuggingFace pipeline for easy model inference
2■
3■ # Load the sentiment analysis pipeline # Comment: load the sentiment-analysis pipeline
4■ sentiment_analyzer = pipeline("sentiment-analysis") # Load default sentiment model (DistilBERT
fine-tuned on SST-2 dataset)
5■
6■
7■ def analyze_sentiment(text): # Define function that runs the model and returns a formatted result
string
8■     result = sentiment_analyzer(text) # Pass the input text through the sentiment model
9■     label = result[0]['label'] # Extract predicted label: 'POSITIVE' or 'NEGATIVE'
10■     score = result[0]['score'] # Extract confidence score as a float between 0 and 1
11■     return f"Sentiment: {label} (Confidence: {score:.2f})" # Return a formatted string showing label
and score to 2 decimal places
12■
13■
14■ while True: # Start an infinite loop to keep accepting user input
15■     user_input = input("Enter a sentence for sentiment analysis (or 'exit' to quit):").strip() # Read
a sentence from the user and remove leading/trailing whitespace
16■     if user_input.lower() == 'exit': # Break out of the loop when user types 'exit'
17■         break # Otherwise run sentiment analysis and print the result
18■     print(analyze_sentiment(user_input))
```


Lab 7: Text Summarisation with BART

Loads facebook/bart-large-cnn and summarises a long air-pollution article using beam search.

```
1 from transformers import AutoModelForSeq2SeqLM, AutoTokenizer # Import AutoModelForSeq2SeqLM and
AutoTokenizer from HuggingFace transformers
2
3 model_name = "facebook/bart-large-cnn" # Specify the BART model fine-tuned for summarisation on
CNN/DailyMail dataset
4 tokenizer = AutoTokenizer.from_pretrained(model_name) # Download and initialise the BART tokeniser
from HuggingFace Hub
5 model = AutoModelForSeq2SeqLM.from_pretrained(model_name) # Download and load the BART seq2seq model
weights into memory
6
7 text = ""Air pollution is the presence of substances in the air that are harmful # Begin defining the
long input article as a multi-line string
8 to humans, other living beings or the environment. Pollutants can be gases, like
9 ozone or nitrogen oxides, or small particles like soot and dust. Both outdoor and
10 indoor air can be polluted. Outdoor air pollution comes from burning fossil fuels
11 for electricity and transport, wildfires, some industrial processes, waste management,
12 demolition and agriculture. Indoor air pollution is often from burning firewood or
13 agricultural waste for cooking and heating. Other sources of air pollution include
14 dust storms and volcanic eruptions. Many sources of local air pollution, especially
15 burning fossil fuels, also release greenhouse gases that cause global warming. However,
16 air pollution may limit warming locally. Air pollution kills 7 or 8 million people
17 each year. It is a significant risk factor for a number of diseases, including stroke,
18 heart disease, chronic obstructive pulmonary disease (COPD), asthma, coronavirus and
19 lung cancer. Particulate matter is the most deadly, both for indoor and outdoor pollution.
20 Ozone affects crops, and forests are damaged by the pollution that causes acid rain.
21 Overall, the World Bank has estimated that welfare losses (premature deaths) and
22 productivity losses (lost labor) caused by air pollution cost the world economy over
23 $8 trillion per year. Various technologies and strategies reduce air pollution. Key
24 approaches include clean cookers, fire protection, improved waste management, dust
25 control, industrial scrubbers, electric vehicles and renewable energy. National air
26 quality laws have often been effective, notably the 1956 Clean Air Act in Britain and
27 the 1963 US Clean Air Act. International efforts have had mixed results: the Montreal
28 Protocol almost eliminated harmful ozone-depleting chemicals, while international action
29 on climate change has been less successful. ""
30
31 # Tokenize the input text # Comment: tokenize the input text
32 inputs = tokenizer.encode(text, return_tensors="pt", max_length=512, truncation=True) # Encode
article to PyTorch tensor, truncate to BART's 512-token limit
33
34 # Generate the summary # Comment: generate the summary
35 summary_ids = model.generate(inputs, max_length=50, min_length=25, # Run beam-search generation: 4
beams, output capped between 25-50 tokens
36 length_penalty=2.0, num_beams=4, early_stopping=True) #
length_penalty>1 encourages longer summaries; early_stopping halts when all beams finish
37
38 summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True) # Decode the generated token IDs
back to a readable string, removing special tokens
39
40 print(summary) # Print the final summarised text to stdout
```

Lab 9: Wikipedia Institution Scraper with Pydantic

Fetches a Wikipedia page, extracts founder/year/branches/employees via regex, and validates with Pydantic.

```
1 import re # Import re module for regular expression pattern matching
2 import wikipediaapi # Import wikipedia-api for programmatic Wikipedia page access
3 from pydantic import BaseModel, Field # Import BaseModel and Field from Pydantic for data validation
4 from typing import List, Optional # Import List and Optional type hints
5
6
7 # --- Data Schema ---
8 class InstitutionDetails(BaseModel): # Define Pydantic data model representing one institution's
    details
9     name: str # Required string field for the institution's name
10    founder: Optional[str] = None # Optional founder field; defaults to None if not found
11    founded: Optional[str] = None # Optional founding year as string; defaults to None
12    branches: List[str] = Field(default_factory=list) # Optional list of branch locations; defaults
    to empty list
13    number_of_employees: Optional[int] = None # Optional integer for employee count; defaults to None
14    summary: Optional[str] = None # Optional 500-character summary snippet from Wikipedia
15
16
17 # --- Extraction Logic ---
18 def fetch_institution_details(institution_name: str) -> InstitutionDetails: # Main extraction
    function; returns a validated InstitutionDetails object
19     # 1. Setup Wikipedia API with a proper User-Agent # Comment: setup Wikipedia API with a proper
    User-Agent
20     user_agent = "InstitutionScraper/1.0 (contact: myemail@example.com)" # Set a descriptive
    User-Agent string as required by Wikipedia's API policy
21     wiki = wikipediaapi.Wikipedia(user_agent=user_agent, language='en') # Initialise Wikipedia API
    client for English language pages
22     page = wiki.page(institution_name) # Fetch the Wikipedia page object for the given institution
    name
23
24     if not page.exists(): # Raise a ValueError if the page does not exist on Wikipedia
25         raise ValueError(f"The page for '{institution_name}' does not exist.")
26
27     full_text = page.text # Store the full plain-text content of the Wikipedia article
28
29     # 2. Helper function to extract info using Regex
30     def extract_pattern(pattern, text, is_list=False): # Comment: helper function to extract info
    using regex
31         match = re.search(pattern, text, re.IGNORECASE) # Run a case-insensitive regex search on the
    article text
32         if match: # Extract the first capture group from the match object
33             content = match.group(1).strip()
34             if is_list: # If is_list=True, split the match on commas to produce a list
35                 return [item.strip() for item in content.split(',')]
36             return content # Return the matched string (or None if no match)
37         return [] if is_list else None # Return empty list or None depending on is_list flag
38
39     # 3. Extraction logic
40     founder_pattern = r"(?:founded|established|started)\s+by\s+([\^.\n,]+)" # Comment: extraction
    logic
41     # Regex pattern to capture the name after 'founded/established/started by'
42     founder_match = re.search(founder_pattern, full_text, re.IGNORECASE)
43     # Run the founder regex search on the full article text
44     founder = founder_match.group(1).strip() if founder_match else "Unknown" # Extract founder name
    or default to 'Unknown'
45
46     year_pattern =
    r"(?:founded|established|started|incorporated)(?:\s+in)?(?:\s+the\s+year)?\s+(\d{4})" # Regex to capture
```

```

a 4-digit year after founding-related keywords
47■
48■     year_match = re.search(year_pattern, full_text, re.IGNORECASE) # Run the year regex search on the
article text
49■
50■     founded = year_match.group(1) if year_match else "Unknown" # Extract founding year string or
default to 'Unknown'
51■
52■     branches = extract_pattern(r"Branches\s*[:\~]?s*(.*)", full_text, is_list=True) # Extract branch
locations as a comma-split list
53■
54■     raw_employees = extract_pattern(r"Number of employees\s*[:\~]?s*([\d,]+)", full_text) # Extract
raw employee count string (may contain commas)
55■     emp_count = None # Initialise employee count as None
56■     if raw_employees: # Only proceed if the regex found an employee number
57■         try: # Remove commas and convert to integer
58■             emp_count = int(raw_employees.replace(',', '')) # Handle case where the extracted string
cannot be parsed as int
59■         except ValueError:
60■             emp_count = None
61■
62■     return InstitutionDetails( # Construct and return the validated Pydantic model with all extracted
fields
63■         name=page.title, # Use the Wikipedia page's official title as the name
64■         founder=founder, # Pass extracted founder string
65■         founded=founded, # Pass extracted founding year
66■         branches=branches, # Pass extracted branches list
67■         number_of_employees=emp_count, # Pass parsed employee count integer
68■         summary=page.summary[:500] + "..." # Attach first 500 chars of the Wikipedia summary with
ellipsis
69■     )
70■
71■
72■ # --- Execution --- # Comment: Execution block
73■ try: # Wrap in try/except to handle pages that don't exist or parse errors
74■     val = input("Enter the Institution name: ") # Prompt user to enter an institution name
75■     data = fetch_institution_details(val) # Call the main scraper function
76■     print(data.model_dump_json(indent=2)) # Pretty-print the Pydantic model as indented JSON
77■ except Exception as e: # Catch any exception and print its message
78■     print(f"Error: {e}") # Print the error to the user

```

Lab 10: IPC PDF Chatbot using PyMuPDF

Extracts text from an IPC PDF and answers keyword queries interactively using line-level search.

```
1 import fitz # PyMuPDF # Import fitz (PyMuPDF) for reading and parsing PDF files
2
3
4 def extract(file): # Define extract(): reads every page of a PDF and returns all text
5     """Extract text from a PDF file.""" # Docstring: Extract text from a PDF file
6     text = "" # Initialise empty string to accumulate text from all pages
7     with fitz.open(file) as pdf: # Open the PDF safely using a context manager (auto-closes on exit)
8         for page in pdf: # Iterate over every page object in the PDF document
9             text += page.get_text() # Extract plain text from the page and append to the result string
10    return text # Return the complete concatenated text of the entire PDF
11
12
13 def search(query, ipc): # Define search(): finds lines containing the query keyword
14     """Search for a query in the IPC document.""" # Docstring: Search for a query in the IPC document
15     query = query.lower() # Lowercase the query for case-insensitive comparison
16     lines = ipc.split("\n") # Split the full document text into individual lines
17     results = [line for line in lines if query in line.lower()] # Keep only lines where the
lowercased query appears
18
19     if results: # If matches found, return at most 15 lines to avoid overwhelming output
20         return results[:15]
21     else:
22         return ["No relevant section found."] # Return a fallback message when no matching lines are
found
23
24
25 # Step 3: Main Chatbot Function
26 def chatbot(): # Define chatbot(): the main interactive query loop
27     """Interactive chatbot for IPC document queries.""" # Docstring: Interactive chatbot for IPC
document queries
28     print("Loading IPC document...") # Notify user that the PDF is being loaded (may take a moment)
29     ipc = extract(r"/Users/ananth/Desktop/ipc.pdf") # Extract all text from the IPC PDF file at the
given path
30
31     while True: # Start an infinite loop to accept repeated user queries
32         query = input("Ask a question about the IPC: ") # Prompt the user to type a question or
keyword
33         if query.lower() == "exit": # If the user types 'exit', break out of the loop
34             print("Goodbye!") # Print a goodbye message on exit
35             break
36
37         results = search(query, ipc) # Search the extracted IPC text for the user's query
38         print("\n".join(results)) # Print all matching lines joined by newlines
39         print("-" * 50) # Print a separator line between consecutive query results
40
41
42 chatbot() # Entry point: call chatbot() to start the interactive session
```